

Regressional NLP and Proximal Policy Optimization

Naowal Rahman

1 Introduction

The release of ChatGPT has produced a newfound public interest in AI and machine learning. ChatGPT uses natural language processing (NLP) to respond to prompts in ways that are optimized for conversation. In this article, we'll first look at traditional algorithms that can be used to train a chat bot and then dive into how ChatGPT was trained. We'll discuss logistic regression, how that can be extended to softmax regression, and the gist of proximal policy optimization, a new family of algorithms that OpenAI uses.

2 An Overview of Neural Networks

In case you aren't familiar with neural networks or need a review, here is an overview of how they work.

Neural networks produce an output given some input. They consist of groups of neurons called layers. Each layer serves a different purpose. Everything starts at the input layer, where input variables are passed. Each neuron in this layer will represent one input variable. If we want to pass in multiple inputs, we need multiple neurons. The next set of layers are called hidden layers. Here, weights are applied to the inputs and are passed through an activation function, which produces outputs that introduce non-linearity to the data. Each layer is fit to produce a specific type of output, and subsequent hidden layers can work in conjunction and use a previous layer's data to produce new output.

2.1 Exclusive-Or Classification Model

To further clarify, let's look at an example. Using Tensorflow's neural network visualizer we can see all of the elements of a neural network aimed at learning an exclusive-or (xor) function, which returns 1 if either x_1 or x_2 is true, and false otherwise.

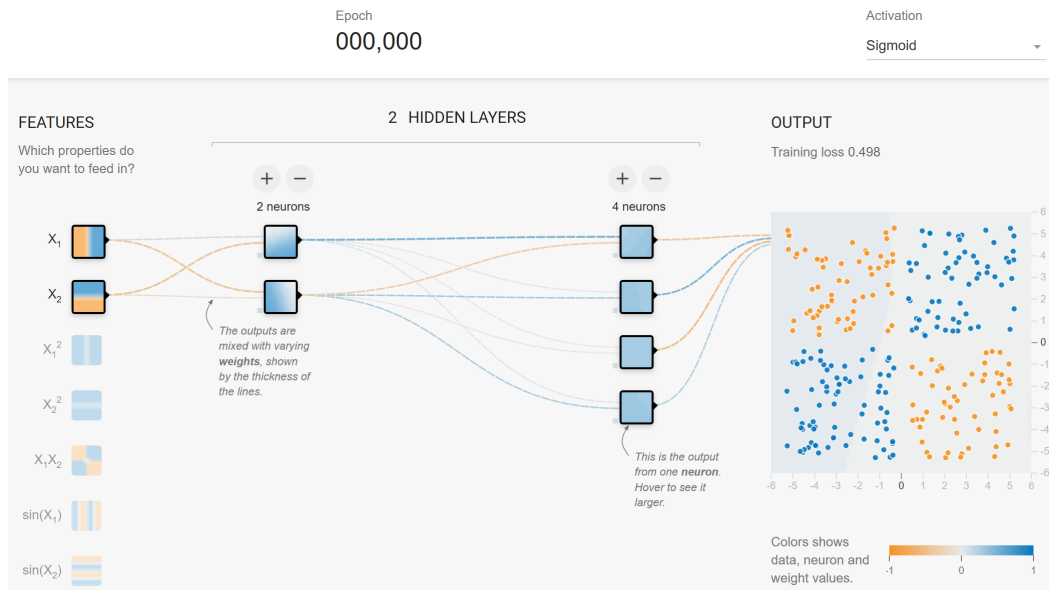


Figure 1: Setup for the xor neural network

Input variables x_1 and x_2 are passed, as those are the two variables used in the xor function. While any number of hidden layers and neurons within those layers can be used, two hidden layers work well, one with two neurons and another with four neurons. This is because we expect our graph to be split into around four sections, with the first hidden layer splitting the graph in half and the second layer distinguishing further from there.

We can see how the model starts off with a loss value close to 1 (meaning high error) and the graph covered entirely around white (0). The neural network starts off with this set of inaccurate values and improves them over time.

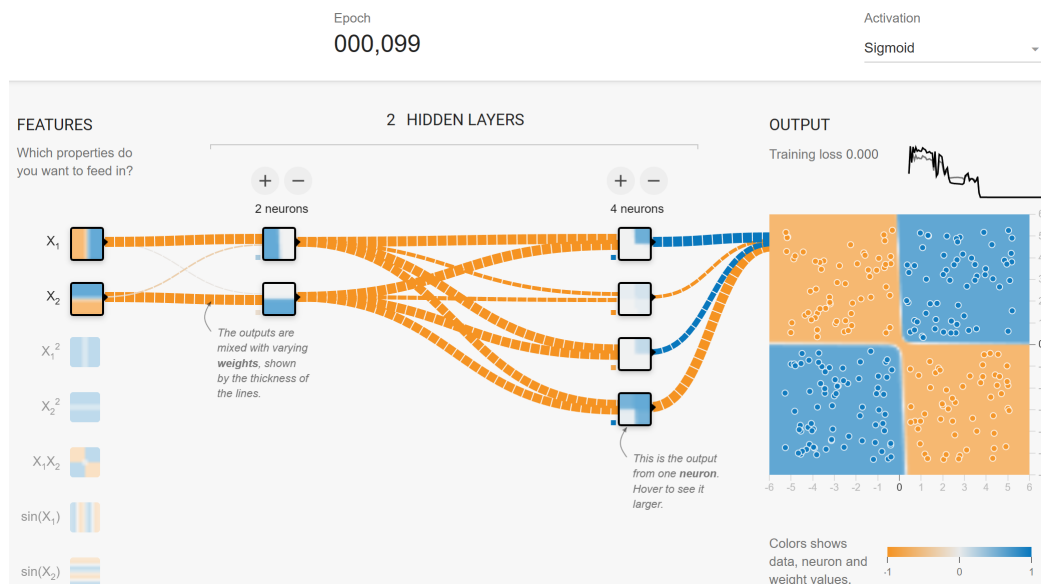


Figure 2: Trained xor model

As expected, this configuration of neurons worked well and the model was trained with a final loss of 0.

3 Logistic Regression

Generally, an NLP algorithm that produces the next most probable output will have to make choices between 2 or more possible outputs. The output is generally an array of probabilities summing to 1 – we pick the index with the highest probability as our model’s choice.

3.1 Sigmoid Function

Not directly used in NLP but essential to generating a probability based on a single input is the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

This is perfect for generating probabilities, as $\sigma(x) \in (0, 1)$, and $x \in (-\infty, \infty)$. No matter our real-number input, the output will always be suited to be a probability. Graphing the function, we see the familiar S-curve, where $\sigma(x) \rightarrow 0$ as $x \rightarrow -\infty$ and $\sigma(x) \rightarrow 1$ as $x \rightarrow \infty$.

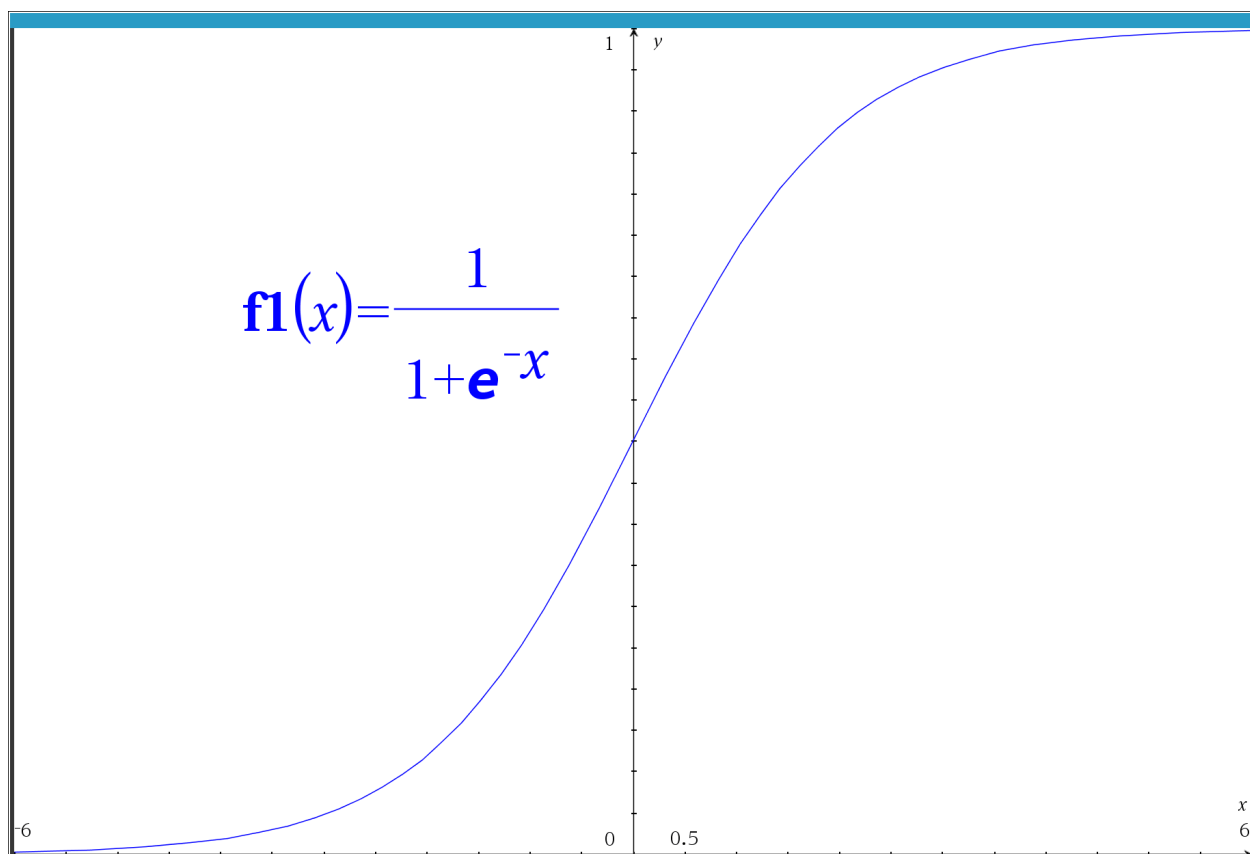


Figure 3: Graph of $\sigma(x)$

This is frequently used to add non-linearity to a model, where the function operates on each of the available inputs and corresponding weights.

3.2 Extending σ to a Multi-Input Scenario

Using σ , we arrive at logistic regression, a basic model that may be used in an NLP program. It generates probabilities for a binary outcome given multiple inputs, and is defined as

$$\begin{aligned} p(y = 1|x) &= \frac{1}{1 + e^{-(b+w_1x_1+w_2x_2+\dots+w_nx_n)}} \\ &= \frac{1}{1 + e^{-(\mathbf{w}^T \cdot \mathbf{x} + b)}} \end{aligned}$$

where

1. $x_1, x_2, \dots, x_n$ are the independent variables.
2. $w_0, w_1, w_2, \dots, w_n$ are the model's weights.
3. We simplify with linear algebra and thus take the dot product of the transpose of the weight vector $\mathbf{w}^T$ and the input vector $\mathbf{x}$.
4. b is the bias term.

Given x , logistic regression produces the probability that $y = 1$ (y is true).

While this isn't sufficient for a chat bot, we can use this for determining properties of text, like whether a sentence is positive or negative.

3.2.1 Exercise

Let $z = \mathbf{w}^T \cdot \mathbf{x} + b = \ln(a)$. Find a .

3.2.2 Solution

Substituting z into $p(x)$, we get

$$p(x) = y = \frac{1}{1 + e^{-(\mathbf{w}^T \cdot \mathbf{x} + b)}} = \frac{1}{1 + e^{-z}}$$

meaning we can solve for z . So, we have

$$\begin{aligned} y &= \frac{1}{1 + e^{-z}} \\ \Rightarrow 1 + e^{-z} &= \frac{1}{y} \\ \Rightarrow -z &= \ln\left(\frac{1}{y} - 1\right) = \ln\left(\frac{1-y}{y}\right) \\ \Rightarrow z &= -\ln\left(\frac{1-y}{y}\right) = \ln\left(\left(\frac{1-y}{y}\right)^{-1}\right) = \ln\left(\frac{y}{1-y}\right) \end{aligned}$$

$$\therefore \boxed{a = \frac{y}{1-y}}$$

3.2.3 Remark

This means that z , the input to $p(x)$, can be represented as the natural logarithm of the probability that y is true divided by the probability that y is false. Thus, z is referred to as the *log-odds ratio*, or $\text{logit}(y)$.

4 Softmax Regression

As logistic regression produces a probability for a binary outcome, softmax regression extends this idea into multi-class scenarios. Thus, rather than returning a single probability $p \in [0, 1]$, softmax returns a vector $\mathbf{p}$ of n probabilities such that $\sum_{i=1}^n p_i = 1$.

4.1 Implementation

We can implement this by adding an extra hidden layer just before the output layer. This hidden layer must have the same number of neurons as the output layer, or the number of classes. Each of these neurons produces a logit value for their respective class to be passed into the sigmoid function that produces probabilities in the output layer.

Mathematically, we represent this as an extension of the logistic regression equation:

$$\begin{aligned} p(y = j|\mathbf{x}) &= \frac{e^{(w_j^T \mathbf{x} + b_j)}}{\sum_{k=1}^K e^{(w_k^T \mathbf{x} + b_k)}}, \quad \text{for } j = 1, 2, \dots, K \\ &= \frac{e^{\text{logit}(y_j)}}{\sum_{k=1}^K e^{\text{logit}(y_k)}}, \quad \text{for } j = 1, 2, \dots, K \end{aligned}$$

Thus, each probability y_j is calculated as the ratio of class j 's exponentiated logit score over the sum of all K classes' exponentiated logit scores.

4.2 Caveat

Softmax regression can only be used under the assumption that all input belongs to exactly one class. This is because the highest probability in the generated output vector is considered the choice; if

$$\mathbf{x} = [0.25, 0.12, 0.15, 0.31, 0.17]$$

then it makes sense to take only the index of $\max(\mathbf{x})$ as the model's choice.

5 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is OpenAI's new class of reinforcement learning algorithms. They are useful because they perform better than current approaches whilst being simpler to implement and modulate.

5.1 Types of Machine Learning

PPO differs distinctly from softmax regression because they are classified as different types of learning algorithms. It is important to know the differences between them before proceeding, so here are the 3 types of learning algorithms: supervised, unsupervised, and reinforcement.

5.1.1 Supervised Learning

Supervised learning algorithms use labeled datasets to make a mathematical model for future datasets. Such algorithms are used for classification problems because they can produce a discrete value, deciding the specific class that the input belongs in. They may also be used for regression problems, where discrete relationships between data need to be created.

Softmax regression, being a classification problem, is a supervised learning algorithm.

5.1.2 Unsupervised Learning

In contrast to supervised learning, unsupervised learning involves the use of *unlabeled* data. Essentially, such algorithms find patterns in data without being trained in a strict input-output manner. They are used for problems where datasets need to be clustered and analyzed.

5.1.3 Reinforcement Learning

Reinforcement learning algorithms are not deliberately trained. Rather, they are continuously training themselves with data from their environment. You may see why this is good for NLP problems or ChatGPT - as more users use the service, the model becomes better and better. Other use cases can include gaming or self-driving cars, scenarios where there are continuous streams of data.

5.2 Training ChatGPT

ChatGPT was trained using both supervised and reinforcement learning. The model was improved over GPT-3.5, an improved version of GPT-3. First, in supervised learning, human trainers provided the model with responses resembling those from both the user and the model. Then, in the reinforcement learning step, the model generated responses that were rated by the human trainers. The ratings were used to further improve the model through PPO 2.

5.3 Explanation

PPO is used to train policies for decision-making problems. This means that the objective function must maximize some expected "reward" obtained from following a given policy over a trajectory of states and actions. A good example of this is that of self-driving cars – the states involve the properties of car at a given timestamp, like velocity and acceleration, and the actions involve what is done by the car at those states, like steering at a certain angle or braking. The associated reward may be related to trip time or energy used.

Expressing this computationally means using a clipped surrogate objective function that limits how the policy is updated to a trust region of allowable updates. We define this trust region through a hyperparameter (a parameter external to the model's control) called a "clipping parameter" to the model. Then, we must factor in the loss of our model and an entropy bonus term that encourages the model to explore different actions.

5.4 Implementation

We can express our clipped surrogate objective as follows:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where

1. θ is the policy parameter.
2. $\hat{\mathbb{E}}_t$ is the average empirical expectation at timestamp t , taken from a batch of finite samples.
3. r_t is the ratio of the old probability to the new probability.
4. $\hat{A}_t$ is the estimate of the current advantage.
5. ϵ is the clipping parameter.
6. $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t$ clips $r_t(\theta)$, keeping it between $[1 - \epsilon, 1 + \epsilon]$.

Then, we can define our squared-error loss function:

$$L^{\text{VF}}(\theta) = \left(V_\theta(s_t) - V_t^{\text{targ}} \right)^2$$

where

1. V_θ is the value estimate predicted by the policy with parameter θ .
2. s_t is the state in question at timestamp t .
3. V_t^{targ} is the current target value.

Finally, we have our entropy bonus S . For some policy and state configuration, the entropy bonus is denoted as $S[\pi_\theta](s_t)$, where π_θ is the policy.

Combining these 3 terms, we get our full objective function:

$$L_t(\theta) = \hat{\mathbb{E}} \left[L_t^{\text{CLIP}}(\theta) - c_1 L_t^{\text{VF}}(\theta) + c_2 S[\pi_\theta](s_t) \right]$$

where c_1 and c_2 are coefficients. Using stochastic gradient descent, this objective can be tuned with different hyperparameters and optimized, finding the best policy.

6 Conclusion

We have covered a wide variety of machine learning algorithms, from ones used in classification problems to those of OpenAI. We started with the foundations of producing probabilities in machine learning using the sigmoid function. From there, we discussed logistic regression, its extension into multi-class scenarios as softmax regression, and onto different types of learning. We looked at the types of learning ChatGPT uses, and dove into how PPO is used, forming a final objective function for a generic PPO algorithm. Each algorithm may be used to create some form of an NLP program, but the applications correspond to the capabilities of each algorithm. While logistic regression can decide a binary outcome, softmax regression decide between multiple classes. On the other hand, PPO, being a reinforcement learning algorithm, can make improvements over

time, making use of the reward-model nature of reinforcement learning.

I hope this article has taught you more about extending common functions to NLP. I also hope you've learned the power of existing learning algorithms and the promise of PPO. As a challenge, try using TensorFlow to program your own chatbot, keeping the equations in mind, or mess around with TensorFlow's neural network playground and get a feel for what's happening. Good luck!

References

ChatGPT. Wikipedia, Dec. 2022.

<https://en.wikipedia.org/wiki/ChatGPT>

Deepak Dilipkumar. States, Actions, Rewards — The Intuition behind Reinforcement Learning. Medium, Mar. 2020.

<https://towardsdatascience.com/states-actions-rewards-the-intuition-behind-reinforcement-learning-33d4aa2bbfaa>

Hidden Layer. DeepAI, May 2019.

<https://deepai.org/machine-learning-glossary-and-terms/hidden-layer-machine-learning>.

John Schulman et al. Proximal Policy Optimization Algorithms. Aug. 2017.

<https://arxiv.org/pdf/1707.06347.pdf>.

Logistic Regression: Calculating a Probability — Machine Learning. Google Developers.

<https://developers.google.com/machine-learning/crash-course/logistic-regression/calculating-a-probability>.

Multi-Class Neural Networks: Softmax — Machine Learning Crash Course. Google Developers.

<https://developers.google.com/machine-learning/crash-course/multi-class-neural-networks/softmax>.

Mehreen Saeed. A Gentle Introduction to Sigmoid Function. Machine Learning Mastery, Aug. 2021.

<https://machinelearningmastery.com/a-gentle-introduction-to-sigmoid-function/>.

Surbhi Arora. Supervised Vs Unsupervised Vs Reinforcement. Aitude, Jan. 2020.

<https://www.aitude.com/supervised-vs-unsupervised-vs-reinforcement/>